FULL-STACK DEVELOPER PORTFOLIO

Portfolio

네 개의 프로젝트를 통한 성장 기록

PROJECT 01
AuRa
FIRST 팀 쇼핑몰 · React + Node.js
4인 팀 · 3주

PROJECT 02
KISETSU
계절 솔로 · React + Socket.io
1인 솔로 · 10일

PROJECT 03
재무제표 자동화
Python + BrityRPA
3인 팀 · 2주

PROJECT 04
개발환기좀해 ERP
Spring Boot + JSP
1인 솔로 · 5주

React · Node.js · Spring Boot · MariaDB · JWT · Socket.io · MyBatis · Python · BrityRPA · Nodemailer

2025 — 2026

INDEX

목차

PROJECT 01 — AuRa / FIRST 팀 쇼핑몰

01 **프로젝트 개요** — AuRa 소개, 기술 스택, 팀 구성 — 03

02 **주요 기여 — DB · 인증 · 장바구니** — 담당 기능 상세, 코드 구현 방식 — 04

03 **기술적 어필 포인트** — 관심사 분리, 중복 방지, 서버단 검증 — 05

PROJECT 02 — KISETSU / 계절 솔로

04 **프로젝트 개요 · 아키텍처** — 기획 배경, 시스템 구조, 기술 스택 — 07

05 **주요 기능 구현** — JWT, 실시간 채팅, Context API — 09

06 **결과 · 회고** — 구현 성과, 배운 점, 성장 스토리 — 10

PROJECT 03 — 재무제표 자동화 (Python + BrityRPA)

07 **프로젝트 개요** — 기획 배경, 시스템 아키텍처, 담당 역할 — 12

08 **담당 역할 — RPA 자동화 파이프라인** — 아키텍처 설계, RPA 구현, 데이터 검증 — 13

09 **핵심 기능 구현** — Python 실행, Excel 차트, 메일 발송 — 14

10 **결과 · 회고** — RPA 성과, 잘된 점, 아쉬운 점 — 15

PROJECT 04 — 개발환기좀해 ERP (Spring Boot + MyBatis)

11 **프로젝트 개요 · 아키텍처** — 기획 배경, 전체 모듈, 기술 스택 — 17

12 **핵심 기능 구현** — OCR, BOM 자동발주, 재무 리포트 — 18

13 **성능 최적화 · DB 쿼리 튜닝** — N+1, 인덱스 최적화 (14.25s→0.17s) — 19

14 **기술적 어필 & 회고** — Spring Security, 트랜잭션, 성과 — 20

PROJECT 01

AuRa

향수 이커머스 쇼핑몰
REST API · 풀스택 팀 프로젝트

유형	기간	역할
4인 팀 프로젝트	약 3주	DB · 인증 · 장바구니 · 카테고리

MariaDB · Node.js · Express · React · Vite · REST API · 커넥션 풀

01 프로젝트 개요

AuRa 향수 쇼핑몰 — 팀 프로젝트

4인	3주	4개	REST
팀 구성	개발 기간	담당 기능	API 설계

서비스 전체 기능

- 회원가입 / 로그인
- 상품 목록 / 상세 / 검색
- 카테고리 필터 & 정렬
- 장바구니 / 위시리스트
- 주문 / 결제 처리
- AI 챗봇 상담
- 미니게임 이벤트

내 담당 기능

- **DB 설계** — MariaDB 커넥션 풀 모듈화
- **로그인 / 회원가입** — REST API, 서버단 검증
- **장바구니** — CRUD API, 중복 방지 로직
- **카테고리** — DB 기반 동적 카테고리, 필터 & 정렬

기술 스택

Database	Backend	Frontend
MariaDB	Node.js, Express	React, Vite

02 주요 기여 — 구현 상세

DB 설계 · 로그인/회원가입 · 장바구니 · 카테고리

DB — 커넥션 풀 모듈화

db.js

```
// MariaDB 커넥션 풀 모듈 분리
const mariadb = require('mariadb')
require('dotenv').config()

const pool = mariadb.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
  connectionLimit: 10
})

module.exports = pool
```

로그인 / 회원가입 API

auth.js

```
// POST /api/auth/login
app.post("/api/auth/login", async (req, res) => {
  const { username, password } = req.body
  // 입력값 누락 시 서버단 검증
  if (!username || !password)
    return res.status(400).json({ msg: "필수값 누락" })
  const [user] = await db.query(
    `SELECT * FROM users
     WHERE username=? AND password=?`,
    [username, password]
  )
  if (!user) return res.status(401).json({...})
  res.json({ success: true, user })
})
```

장바구니 — 중복 방지 핵심 로직

cart.js

```
// POST /api/cart/add
// 핵심: 중복 상품 서버단 검증
app.post("/api/cart/add", async (req, res) => {
  const { userId, productId } = req.body
  const [exist] = await db.query(
    `SELECT * FROM cart
     WHERE user_id=? AND product_id=?`,
    [userId, productId]
  )
  if (exist)
    return res.json({ msg: "이미 담긴 상품" })
  await db.query(
    `INSERT INTO cart VALUES(?,?)`,
    [userId, productId]
  )
})
```

카테고리 — DB 기반 동적 쿼리

category.js

```
// GET /api/category (DB 기반)
app.get("/api/category", async (req, res) => {
  // 하드코딩 없이 DB에서 동적 조회
  const cats = await db.query(
    `SELECT * FROM categories`
  )
  res.json(cats)
})
// 가격 필터 + 정렬 동적 쿼리
let sql = `SELECT * FROM products
  WHERE category_id=?`
if (minPrice) sql += ` AND price≥?`
sql += ` ORDER BY \${sortField}`
```

03

기술적 어필 & 회고

설계 철학과 팀 프로젝트에서의 성장

관심사 분리 (모듈화)

DB 연결 로직을 `db.js` 로 분리해 전체 서버에서 재사용 가능하도록 설계. 유지보수 시 DB 설정 변경이 단 1개 파일만으로 가능.

서버단 중복 방지 로직

장바구니·위시리스트 추가 시 프론트가 아닌 서버단에서 DB 조회 후 검증. 클라이언트 조작에도 데이터 무결성 보장.

동적 카테고리 설계

카테고리를 하드코딩이 아닌 DB 기반으로 관리해 코드 수정 없이 카테고리 추가/삭제 가능. 가격 필터와 정렬도 동적 쿼리로 지원.

.env 보안 분리

DB 접속 정보를 `.env` 로 분리해 소스 코드에 직접 노출되지 않도록 보안 강화. 커넥션 풀로 DB 성능 최적화.

팀 프로젝트에서 배운 것

첫 팀 프로젝트였던 만큼 기술적인 어려움뿐 아니라 협업 자체의 어려움도 크게 느꼈습니다. 초반에는 각자 개발한 기능이 서로 맞지 않거나, 말로는 이해한 것 같았지만 실제 구현 결과가 달라 수정이 반복되는 상황이 생겼습니다. **의사소통이 충분하지 않으면 개발 속도보다 수정 비용이 더 커진다**는 것을 몸소 경험했습니다. 이 경험은 이후 KISETSU 솔로 프로젝트에서 설계를 철저히 하는 원동력이 되었습니다.

PROJECT 02

KISETSU

도매 패션 이커머스 플랫폼
실제 운영자 요구 기반 · 1인 풀스택 솔로 프로젝트

유형	기간	규모
1인 솔로 프로젝트	2026.01.15 – 01.30 (2주)	고객 기능 8개 + 관리자 기능 8개

React 19 · Node.js · Express 5 · MariaDB · JWT · Socket.IO · Chart.js · bcrypt · Context API

04 프로젝트 개요 · 아키텍처

KISETSU — 실제 도매업 운영자의 요구를 반영한 플랫폼

개발 목적

- 실제 도매업 운영자의 **온라인 판매 채널 부재 문제** 해결
- 재고·주문·고객 관리를 **하나의 플랫폼으로 통합**
- 오프라인 의존도를 낮추고 **운영 효율화** 도모
- 운영자(지인)와 직접 대화하며 요구사항 수집

주요 기능 목록

- 고객: 회원가입/로그인(JWT) · 상품목록 · 장바구니 → 주문
- 고객: 자동완성 검색 + 인기검색어
- 고객: 실시간 1:1 채팅 (Socket.IO)
- 고객: 계절별 테마 UI (봄/여름/가을/겨울)
- 관리자: 대시보드 · 상품/주문/회원 관리
- 관리자: Chart.js 매출 통계

시스템 아키텍처

Frontend →	React 19 + Vite	고객 화면 · 관리자 화면
통신 →	HTTP (Axios) REST API	WebSocket (Socket.IO)
Backend →	Node.js + Express 5	/auth · /products · /orders · /admin · /chat
Auth →	JWT 미들웨어 · bcrypt 해싱	역할 기반 접근제어
Database →	MariaDB · 커넥션 풀	트랜잭션 처리

05 주요 기능 구현

JWT 인증 · Socket.IO 채팅 · Context API · 인기검색어

JWT 인증 미들웨어

authMiddleware.js

```
const authMiddleware = (req, res, next) => {
  try {
    const token = req.headers
      .authorization?.split(" ")[1]
    if (!token)
      return res.status(401).json({
        msg: "인증 토큰 없음"
      })
    const decoded = jwt.verify(
      token, process.env.JWT_SECRET
    )
    req.user = decoded
    next()
  } catch {
    res.status(403).json({ msg: "유효하지 않은 토큰" })
  }
}
```

SOCKET.IO 실시간 채팅

Chat.jsx

```
// 1단계: 채팅 열릴 때 소켓 연결
useEffect(() => {
  if (isOpen && isLoggedIn) {
    const newSocket = io(SOCKET_URL)
    setSocket(newSocket)
    return () => newSocket.disconnect()
  }
}, [isOpen, isLoggedIn])

// 2단계: 소켓 생기면 이벤트 구독
useEffect(() => {
  if (!socket) return
  socket.on('message', (msg) => {
    setMessages(prev => [...prev, msg])
  })
  return () => socket.off('message')
}, [socket])
```

인기검색어 — SQL 비즈니스 로직

search.js

```
// 최근 7일 주문 기반 인기 상품 추출
const [popular] = await db.query(`
  SELECT p.name,
    COUNT(oi.order_item_id) as order_count
  FROM products p
  JOIN order_items oi ON p.id = oi.product_id
  JOIN orders o ON oi.order_id = o.id
  WHERE o.created_at ≥
    DATE_SUB(NOW(), INTERVAL 7 DAY)
  GROUP BY p.id
  ORDER BY order_count DESC
  LIMIT 5
`)
```

CONTEXT API — 장바구니 전역 상태

CartContext.jsx

```
const CartContext = createContext()

// 커스텀 훅으로 사용처 보호
export const useCart = () => {
  const ctx = useContext(CartContext)
  if (!ctx)
    throw new Error(
      "CartProvider 밖에서 호출됨"
    )
  return ctx
}

export const CartProvider = ({ children }) => {
  const [cart, setCart] = useState([])
  return (
    <CartContext.Provider value={{ cart, setCart }}>
      {children}
    </CartContext.Provider>
  )
}
```

06

결과 · 종합 회고

KISETSU 성과와 두 프로젝트를 통한 성장

8개	8개	100h	실전
고객 기능	관리자 기능	2주 단독 완성	운영자 요구 기반

잘된 점

- 첫 프로젝트 대비 코드 구조와 흐름이 읽히기 시작
- 기획부터 구현까지 전 과정을 혼자 완성
- 실제 운영자 요구를 반영한 서비스 설계 경험
- JWT, Socket.IO, Context API 등 실무 기술 적용
- 풀스택 데이터 흐름 전반 이해

아쉬운 점 / 다음 목표

- 테스트 코드 부재 — 다음 프로젝트에서 도입 목표
- 리팩토링 시간 부족으로 중복 코드 일부 존재
- TypeScript 도입으로 타입 안정성 확보 계획
- CI/CD 파이프라인 구성 학습 중

성장 스토리 — AuRa → KISETSU

AuRa 팀 프로젝트에서 협업의 어려움과 기술적 한계를 경험한 후, KISETSU에서는 **설계를 먼저, 구현은 그 다음**이라는 원칙을 철저히 지켰습니다. DB 테이블 설계, REST API 명세, 컴포넌트 구조를 사전에 정리한 덕분에 2주 안에 고객·관리자 기능 총 16개를 단독으로 완성할 수 있었습니다. 단순히 작동하는 코드를 넘어, **왜 이렇게 설계했는가를 설명할 수 있는 개발자**로 성장하는 것이 목표입니다.

PROJECT 03

재무제표 자동화

Python + BrityRPA · 3인 팀 프로젝트

유형	기간	역할	파이프라인
3인 팀 프로젝트	26.02.28 – 03.10	팀장 · RPA 개발	End-to-End 자동화

Python · BrityRPA · Pandas · openpyxl · SMTP · K-IFRS

07

프로젝트 개요

3인	2주	5개	100%
팀 구성	개발 기간	RPA 자동화	Zero-Touch

개발 목적 & 목표

개발 목적
기업 재무팀 수작업 회계 전표 처리 자동화. 수만 건 Raw Data 비효율 및 휴먼 에러 제거.

목표 ①
Python → 재무 데이터 처리 (ETL)

목표 ②
RPA → 보고서 자동 생성 & 차트

목표 ③
메일 자동 발송 (SMTP)

목표 ④
Raw Data → 재무제표 → 대시보드 → 메일

시스템 아키텍처

Input
CSV Raw Data — 1만 건 가상 분개장

ETL
Python · Pandas → 계정별 집계

Mart
데이터마트 → 재무제표 구조화

RPA
BrityRPA → Excel 차트 자동 구성

Output
SMTP → 경영진 메일 자동 발송

담당 역할 (팀장 · RPA)

아키텍처 설계
Python-RPA Hybrid 역할 분리 구조 설계

인터페이스
error_flag.txt 오류 공유 규격 합의

RPA 구현
Python 실행 · Excel 차트 · 메일 자동화

검증
UI 기반 재무 데이터 정합성 Validation

PM
일정 관리 및 전체 기술 의사결정 리딩

기술 스택

Data Engineering: Python, Pandas, openpyxl
Process Automation: BrityRPA, SMTP
Data Source: CSV Raw Data, K-IFRS

08

담당 역할 — RPA 자동화 파이프라인

팀장으로서 End-to-End 자동화 아키텍처 설계 및 BrityRPA 구현 전담

CSV	Python	Data	BrityRPA	Excel	메일
Raw Data	ETL	Mart	자동화	보고서	발송

① 아키텍처 설계

Hybrid Automation 설계
Python이 연산, RPA가 시각화/배포를 전담하는 역할 분리 구조

인터페이스 규격 수립
Python ↔ RPA 간 데이터 교환 형식 합의. error_flag.txt로 오류 공유

Error Handling
Python 오류 시 플래그 파일 생성 → RPA 자동 중단 설계

파이프라인 리딩
ETL → 데이터마트 → Excel 전체 흐름 총괄

② RPA 자동화 구현

Python 스크립트 실행
BrityRPA ExecuteCmd로 apnalchangchang.py 실행 및 결과 수집

Excel 차트 자동 생성
월별 손익 꺾은선 / 비용구조 파이차트 / 포괄손익계산서 3종

분기별 재무현황 이관
Python 결과 파일 → 보고서 템플릿 시트 자동 복사 · 서식 유지

메일 자동 발송
SMTP 연동, 수신자 자동 설정, Excel 보고서 첨부 발송

③ 데이터 검증 & 관리

재무 데이터 정합성
UI 기반 Validation으로 입력 오류 사전 차단

일정 관리
기획 → Python 개발 → RPA 시각화 → 통합 테스트 단계 리딩

기술 의사결정
Python-RPA 연계 방식, 데이터 마트 구조 결정

팀 협업
타 기술 스택 담당자와 데이터 인터페이스 합의 및 소통

09

핵심 기능 구현

BrityRPA 기반 자동화 · Python-RPA Hybrid Architecture

1. 파이썬 실행 자동화

BrityRPA_ExecuteCmd.bot

BrityRPA의 ExecuteCmd 액션으로 Python 데이터 처리 엔진 실행. RPA가 전체 자동화 파이프라인을 제어하는 Orchestrator 역할.

```
// 1. Python 스크립트 실행
ExecuteCmd(
"python apnalchangchang.py")

// 2. 오류 플래그 감지
IF FileExists("error_flag.txt")
  → RPA 프로세스 중단
```

2. Excel 차트 자동 생성

chart_automation.bot

Python 결과 데이터 기반 Excel Chart Object 자동 생성. 꺾은선 · 파이차트 · 포괄손익계산서 3종 자동화.

```
// 차트 자동화
CreateChart(type="Line")
  .SetDataRange(monthly_pl)
  .SetTitle("월별 손익")
CreateChart(type="Pie")
  .SetDataRange(cost_structure)
  .Export("report.xlsx")
```

3. 분기별 재무현황 이관

data_transfer.bot

Python 생성 재무현황 데이터를 최종 보고서 Excel 템플릿으로 자동 이관. 서식·레이아웃 유지.

```
// 데이터 이관
LoadWorkbook(data_mart.xlsx)
SelectSheet("분기별재무현황")
CopyRange(A1:Z100)
PasteToTemplate(
"보고서템플릿")
MaintainFormatting = True
```

4. 엑셀 저장 & 메일 발송

mail_dispatch.bot

SMTP 서버 연동으로 최종 보고서 자동 배포. 수신자 설정, 메일 본문 작성, Excel 첨부 발송 완전 자동화.

```
// End-to-End 메일 발송
SaveWorkbook("경영대시보드.xlsx")
SMTP.Connect(server, port)
Mail.SetRecipients(list)
Mail.AttachFile(report.xlsx)
Mail.Send()
// 자동 발송 완료
```

10

결과 · 회고

5개	100%	1만 건+	팀장
RPA 자동화 구현	Zero-Touch	전표 처리	E2E 리딩

잘된 점

- 재무자동화 핵심 프로세스 100% 구현
- Python-RPA 하이브리드 아키텍처 완성
- Error Flag 구조로 이기종 시스템 안정 연동
- 원천데이터 파싱 → 메일 발송 Zero-Touch 완성
- 팀장으로 전체 기술 의사결정 및 일정 리딩

△ 아쉬운 점 / 개선 방향

- 초기 원천 데이터 예외 포맷 오류 처리 미흡
- RPA 예외처리 로직 정교화 필요
- 동적 방어 코드 추가 및 에러 로깅 고도화 목표
- 경영진 피드백 기반 대시보드 UI/UX 개선

자체 평가 — 윤종빈

단순한 스크립트 코딩을 넘어, 실제 기업 재무 기준을 시스템 로직으로 완벽히 구현해낸 것이 이번 프로젝트의 가장 큰 성과입니다. Python이 "데이터 연산"을 전담하고 RPA가 "시각화 및 메일 발송"을 담당하도록 아키텍처를 분리하였습니다. 타 기술 스택 담당자와 명확한 데이터 인터페이스 규격을 합의하고, 오류 발생 시 즉각 상태를 공유하는 Error Flag 구조를 설계하며 이기종 시스템 간 매끄러운 연동과 협업 시너지를 제대로 경험할 수 있었습니다.

PROJECT 04

개발환기좀해 ERP

Spring Boot + MyBatis · 1인 솔로 풀스택

유형	기간	규모	특이사항
1인 솔로 프로젝트	26.03.25 – 04.01	모듈 15개+	OCR · AI API 연동

Spring Boot 3.4 · Java 17 · MyBatis · MariaDB · Spring Security · JSP/JSTL · Swagger · Gemini OCR · Lombok

프로젝트 개요 · 전체 모듈

15개+	20개+	1인	5주
컨트롤러/모듈	REST API	솔로 개발	개발 기간

인

인증 · 보안
Spring Security 역할 기반 접근제어
BCrypt 패스워드 해싱
ADMIN / USER 권한 분리

대시보드
실시간 KPI (제품수, 직원수, 매출)
재고부족 알림, 월별 작업지시 통계
최근 주문 · 저재고 품목 리스트

인사 · 근태
직원 CRUD + 사진 업로드
출퇴근 체크인/아웃 API
연차/병가 등록 · 잔여일수 차감

생산 관리
BOM(자재명세서) 관리
작업지시서 생성 · 상태 관리
BOM 기반 재고 부족 시 자동 발주

재고 · 창고
재고 CRUD, 입출고 트랜잭션
창고별 재고 현황 관리
저재고 기준치 설정

구매 · 영업
발주서 워크플로우 (대기→승인→완료)
판매 · 배송 관리
거래처(공급업체) 관리

거래명세서
OCR (Gemini/Groq AI) 자동 인식
이미지 업로드 → DB 자동 등록
인쇄 기능 (거래명세서 출력)

재무 분석
손익계산서 자동 계산 (월별/연별)
영업이익률 · 매출총이익 산출
Python 연동 Excel 리포트 내보내기

관리자
사용자 계정 관리 (ADMIN)
저재고 기준치 시스템 설정
Swagger UI API 문서 자동화

핵심 기능 구현

OCR 자동 인식 · BOM 자동발주 · 재무 리포트 · Spring Security

OCR 거래명세서 자동 인식

OcrController.java

Gemini/Groq AI API 연동. 이미지 업로드 → Python OCR → JSON 파싱 → DB 자동 저장까지 완전 자동화.

```
// 1. 이미지 → Python OCR
ProcessBuilder pb = new
ProcessBuilder(pythonCmd,
  mainPy, "--erp-mode", file);
// 2. JSON 파싱 → DB 저장
stmt.setIssueDate(
  LocalDate.parse(...));
statementService.insert(stmt);
```

BOM 기반 자동 부품 발주

WorkOrderService.java

작업지시 시 BOM 조회 → 재고 부족 자동 감지 → 발주서 자동 생성. 부품 원가 변경 시 완제품 원가 자동 재계산.

```
// 재고 부족 부품 자동 발주
List<BomItem> shortages =
  bomService.checkShortages(
    workOrderId);
for (BomItem item : shortages) {
  purchaseOrderService
    .autoCreate(item);
}  // 발주서 자동 생성
```

재무 분석 · Python Excel

FinancialController.java

매출/비용 데이터 기반 손익계산서 자동 산출. Python 스크립트 연동으로 xlsx 리포트 생성 및 다운로드.

```
// 영업이익률 자동 계산
BigDecimal opMargin =
  opProfit.divide(revenue,
    4, HALF_UP)
  .multiply(
    BigDecimal.valueOf(100));
// Python → xlsx 리포트
byte[] xlsx = runPython(csv);
```

Spring Security 역할 인증

SecurityConfig.java

BCrypt 해싱, 폼 로그인, ADMIN/USER 역할 분리. /admin/** 경로는 ADMIN만 접근 가능.

```
// 역할 기반 접근 제어
.requestMatchers(
  "/admin/**")
  .hasRole("ADMIN")
.anyRequest()
  .authenticated()
// BCrypt 인코더
return new BCryptPasswordEncoder();
```

성능 최적화

성능 최적화 · DB 쿼리 튜닝

BEFORE
14.25 s

→

AFTER
0.17 s
(176 ms)

83x 향상

① N+1 쿼리 문제 해결

BEFORE

```
목록 각 행마다 SUM() 서브쿼리 반복 실행
→ 데이터 100건 = SQL 101번 호출 (과부하)
SELECT *, (SELECT SUM(...)
  FROM items WHERE order_id = o.id)
FROM orders o  -- 행마다 재실행!
```

AFTER

```
파생 테이블로 SUM 미리 집계 후 1회 JOIN
→ 데이터 100건도 SQL 1번으로 처리
SELECT o.*, s.total FROM orders o
LEFT JOIN (SELECT order_id,
  SUM(qty) total FROM items
  GROUP BY order_id) s ON o.id=s.order_id
```

② JOIN 조건 · 인덱스 최적화

BEFORE

```
LIKE 연산자로 customer_name 매칭
→ 인덱스 무효화, 풀 스캔 발생
JOIN suppliers s
  ON o.supplier_info
     LIKE CONCAT('%',s.name,'%')
-- 전체 스캔으로 매우 느림
```

AFTER

```
customer_name 컬럼 직접 참조로 변경
→ 인덱스 즉시 활용, 속도 대폭 향상
JOIN suppliers s
  ON o.customer_name = s.name
-- 인덱스 활용 → 즉시 탐색
-- 14.25s → 0.17s 달성
```

배운 점

"기능 구현"에서 끝나지 않고 실제 사용 환경에서 병목을 직접 발견하고 쿼리 구조를 재설계하여 83배의 성능 향상을 달성했습니다.
N+1 문제와 인덱스 활용의 중요성을 실무 수준으로 체득한 경험입니다.

기술적 어필 & 회고

15개+	100%	OCR	트랜잭션
모듈 구현	1인 완성	AI 연동	원자성 보장

기술적 어필 포인트

- OCR + AI API (Gemini/Groq) 연동 거래명세서 자동 등록
- BOM 기반 재고 부족 자동 감지 → 발주서 즉시 생성
- 입고/출고/생산입고 @Transactional 원자성 처리
- Spring Security 역할 기반 접근제어 + BCrypt
- Python 연동 재무 Excel 리포트 (csv → xlsx 변환)
- 사이드바 아코디언 UI + 현재 메뉴 자동 펼침

△ 아쉬운 점 / 개선 방향

- JSP + JSTL 한계 → 다음엔 React 프론트 분리 목표
- OCR 예외 포맷(누락 · 오타) 처리 로직 보완 필요
- 테스트 코드 없음 → JUnit + MockMvc 도입 계획
- 실시간 알림(WebSocket) 추후 적용 예정

회고 — 개발환기좀해 ERP

제조업 ERP 전 모듈을 1인으로 처음부터 끝까지 구현하면서, 단순한 CRUD를 넘어 실제 업무 프로세스를 코드로 표현하는 경험을 했습니다. BOM 기반 자동 발주, OCR AI 연동, Python 재무 리포트 등 서로 다른 기술 스택을 하나의 시스템으로 통합하는 과정에서 Spring Boot의 계층 구조(Controller-Service-Mapper)와 트랜잭션 관리의 중요성을 몸소 익혔습니다. JSP의 한계를 느끼면서 React 분리의 필요성도 깨달았고, 다음 프로젝트에서는 프론트엔드를 완전히 분리한 REST API 기반 아키텍처를 목표로 합니다.